

Speaker Notes: Metal Segmentation in CT/CBCT

A beginner-friendly guide to explaining both papers

This document walks through the two papers slide-by-slide, explaining the ideas in plain language. Read this alongside the presentation — each section below maps to the matching slide.

Background you need before anything else

What is CT / CBCT? A CT (Computed Tomography) or CBCT (Cone Beam CT) scanner rotates an X-ray source and detector around a patient. At each angle, it takes a 2D X-ray picture called a **projection**. After collecting projections from many angles, a **reconstruction** algorithm (the most common is called FDK — Feldkamp-Davis-Kress) combines them into a 3D image made of voxels (3D pixels), each with a brightness value measured in **Hounsfield Units (HU)**. HU roughly tells you how dense a material is: air is about -1000, water is 0, bone is around 500-1500, and metal is above 3000-4000.

What is a “metal artifact”? Metal (screws, plates, dental fillings, hip implants) is far denser than anything else in the body. X-rays barely pass through it. This confuses the reconstruction math and creates bright/dark streaks, shadows, and blooming (metal looking bigger than it really is) around the metal in the final image. These artifacts can hide real anatomy from doctors.

What is “metal segmentation”? Segmentation just means: draw a mask (a black-and-white image) where white = “this is metal” and black = “this is not metal.” Once you know exactly where the metal is, you can remove/replace that region in the data and reconstruct a cleaner image. This process of fixing the image is called **Metal Artifact Reduction (MAR)**.

Two places you can do this segmentation: - **Image domain** = on the final reconstructed 3D image (the slices doctors look at, in HU values). - **Projection domain** = on the raw 2D X-ray pictures, *before* reconstruction happens.

Both papers you’re presenting tackle the same underlying problem (segmenting metal) but in these two different “spaces.” That’s the core comparison your presentation is making.

What is U-Net? U-Net is a very famous neural network shape for segmentation tasks (from a 2015 paper, also attached to your files). It looks like the letter “U”: it shrinks the image down through several layers (encoder, learns *what* is in the image), then expands it back up to full size (decoder, recovers *where* things are). “Skip connections” copy detail from the shrinking path directly to the expanding path so fine edges aren’t lost. Almost every model in both papers is some variant of U-Net.

What is “Dice Similarity Coefficient (DSC)” and “IOU”? These are the two main scores both papers use to grade how good a predicted mask is compared to the true mask. - **DSC** = $2 \times (\text{overlap}) / (\text{size of prediction} + \text{size of truth})$. 100% = perfect overlap. - **IOU (Intersection over Union)** = $\text{overlap} / (\text{everything covered by either mask})$. Always a bit lower than DSC for the same prediction. Higher is better for both. You’ll see these numbers everywhere — just remember: *they measure how well the predicted white blob matches the true white blob.*

Slide 1 — Title

Just introduce the topic: you reviewed two recent research papers that both try to solve the same problem (segmenting metal for artifact reduction) but in two different “domains” — and you also did a small hands-on project (the notebook) that fits into this same area.

Slide 2 — Agenda

Simple roadmap slide. Say: “I’ll explain why this problem matters, the two possible approaches, then go through each paper, compare them, and finally show my own implementation.”

Slide 3 — The Problem: Metal Artifacts in CT/CBCT

What to say: “When a patient has a metal implant — like a hip replacement, dental filling, or spinal screw — the CT scanner struggles to image through that metal properly. This creates bright streaks and dark shadows in the final image, which can hide bone and tissue detail near the implant. To fix this (called MAR — Metal Artifact Reduction), the standard pipeline is: scan the patient, **figure out exactly where the metal is** (segmentation — this is the hard, important step both papers focus on), remove/replace that region in the raw data, then reconstruct a corrected image. If the segmentation step is wrong — either marking too much as metal or too little — you either lose real anatomical detail or leave artifacts behind. That’s why getting this step right matters so much, and why both papers use deep learning to do it better than old-school methods.”

Why manual segmentation doesn’t work: A single CBCT scan can have 300-450 separate 2D projection images. A human manually drawing the metal outline on each one for every patient is far too slow and inconsistent — hence the need for automated (deep learning) methods.

Slide 4 — Two Segmentation Domains

This is the key conceptual slide — take your time here.

Image Domain (left card): “Once the scanner has finished and produced the final 3D image (the kind of slice a radiologist looks at), you can look at that image and say ‘these bright voxels above some threshold are probably metal.’ That’s image-domain segmentation. It’s intuitive because you’re working with the final picture. The downside: if you need to use this metal information to fix the *raw* projection data (for projection-based MAR), you have to mathematically project this 3D mask back into all the 2D projection images — an extra, error-prone step called ‘forward projection.’”

Projection Domain (right card): “Instead, you can try to find the metal directly in the raw 2D X-ray projections, *before* any reconstruction happens. This skips the forward-projection step entirely. It’s harder because the projections look very different from a normal image (lots of noise, overlapping shadows), but if it works, it’s more direct and more robust — for example, if a piece of metal is partly outside the scanner’s field of view, the 3D reconstruction might not ‘see’ it at all, but it’s still visible in the raw projections.”

One-line summary for this slide: “Paper 1 works in the projection domain (on CBCT). Paper 2 works in the image domain (on CT). That’s the main axis of comparison in this presentation.”

Slide 5 — Paper 1 Overview (Projection Domain, CBCT)

Paper: “Deep Learning Based Projection Domain Metal Segmentation for Metal Artifact Reduction in Cone Beam Computed Tomography” — Agrawal, Hietanen & Sarkka (Aalto University & Planmeca), IEEE Access 2023.

The core problem they solve: To train a deep learning model to find metal in raw CBCT projections, you need lots of training examples: a projection image paired with the *correct* metal mask for that exact image. But manually drawing these masks on hundreds of projections per scan is extremely slow, and the boundaries of metal are blurry and hard to draw by hand. So — how do you get enough good training data?

Their solution — “fake” the training data using physics

simulation: 1. They start with 10 real CBCT scans that already contain metal. 2. They reconstruct a normal 3D image from each scan and use a simple brightness threshold to roughly find where the metal is in 3D. 3. They clean up this rough metal shape (smoothing, filling holes). 4. **Key idea — Monte Carlo simulation:** They use a physics simulator that mimics how individual X-ray photons travel through a 3D volume, scatter, and get absorbed (this is the same kind of simulation used in nuclear physics / radiation research). Feeding their 3D volumes (with and without the metal) through this simulator produces brand new, *realistic* 2D projection images — including a projection of “metal only,” which becomes the perfect ground-truth mask. 5. This way, they generate thousands of (projection, correct mask) pairs without a human ever drawing a single mask by hand.

“Noisy” vs “Clean” simulations — why this matters: Real X-ray detectors are noisy because only a limited number of photons hit each pixel. Simulating with very few photons (300-1400 per pixel) is fast but produces grainy/noisy images — closer to how real scans actually

look. Simulating with many photons (5000 per pixel) produces clean, smooth images but takes much longer to compute. Surprisingly, **the model trained on the noisier (faster, cheaper) simulations performed just as well or better** — good news, because it's much cheaper to generate this training data.

The model itself: A “modified U-Net” — same overall U-shape as the original U-Net paper, but with some tweaks (instance normalization, bilinear upsampling). Nothing exotic — the *novelty* of this paper is the *training data generation method*, not the network architecture itself (they say this directly).

The “full + crops” trick: They train the network twice over — once on the whole projection image, and once on small zoomed-in crops around the metal. Training on both together made the model both accurate on the metal itself AND less likely to produce false alarms (“false positives”) on scans that have no metal at all.

Headline numbers (Slide 5 stat cards): - **94.8% mean DSC** — their best model (called “Noisy full+crops”) matched the human-drawn ground truth almost perfectly across 10 real test scans. - **90.2% mean IOU** — same model, the other overlap metric. - **90.5% mean DSC for “CMAR”** — this is the old-fashioned baseline method (threshold the 3D image, then forward-project it). Their deep learning model beats it by about 4 percentage points. - **0.51×10^{-3} false positive rate** — on 6 scans that had *no metal at all*, their best model almost never wrongly marked anything as metal. This shows the model generalizes well and isn't just “seeing metal everywhere.”

Takeaway sentence: “Even though they never used a single real hand-labeled training image, training on physics-simulated data alone produced a model that beat the traditional method and worked reliably on real clinical scans — including tricky cases like patient motion and metal sitting outside the visible scan area.”

Slide 6 — Paper 1 Model & Training Pipeline

This slide is the step-by-step “how it's actually built,” for anyone asking technical detail.

Step 1 — Reconstruct & Threshold: Take a real CBCT scan, reconstruct it into a 3D image using FDK (the standard reconstruction algorithm — also one of the references in this paper). Then mark any voxel above ~3400 HU as “metal” (remember: metal is much denser/brighter than bone). Clean this rough mask up with simple image-processing filters (median filter to smooth noise, dilation to fill small gaps).

Step 2 — Monte Carlo Simulation: As explained above — simulate X-ray photons traveling through (a) the volume with metal removed, and (b) the volume with just the metal, to generate a realistic 2D projection and its matching ground-truth metal mask.

Step 3 — Modified U-Net Training: Feed these simulated projections (both full-size and cropped versions) into the U-Net. The network has 4 “down” stages (compressing the image, doubling the number of feature channels each time: 16 -> 32 -> 64 -> 128 -> 256)

and 4 “up” stages (expanding back to full resolution). It’s trained with “binary cross-entropy loss” — a standard way to measure how wrong a yes/no (metal/not-metal) prediction is for every pixel.

Step 4 — Binary Metal Trace Output: The network outputs a probability (0 to 1) for every pixel — “how likely is this pixel to be metal?” Anything above 0.5 probability gets turned into a final white/black (1/0) mask. This mask is the direct output that downstream MAR algorithms use.

Bottom black bar — extra technical detail: They also used standard data augmentation tricks (flipping the image, slightly rotating it, adding random noise, etc.) during training. This is a common practice to make models more robust without needing more real data — it’s also discussed in the original U-Net paper attached to your files.

Slide 7 — Paper 2 Overview (Image Domain, CT)

Paper: “Metal implant segmentation in CT images based on diffusion model” — Xie et al. (Nanjing Medical University), BMC Medical Imaging 2024. Their model is called **DiffSeg**.

What is a “diffusion model”? This is a newer type of AI model (the same family of technology behind AI image generators like Stable Diffusion). The basic idea: 1. **Forward process:** Take the correct answer (here, the true metal mask) and slowly add random noise to it, step by step, until it becomes pure random noise (1000 steps in this paper). 2. **Reverse process:** Train a neural network to *undo* this — starting from random noise, gradually remove the noise, step by step, until a clean mask appears. The network is told what the original CT image looks like (this is the “conditioning”) so it knows what shape to “denoise” toward.

In short: instead of directly predicting the mask in one shot (like U-Net does), DiffSeg gradually *refines* a noisy guess into a clean mask over many small steps. This iterative refinement is often better at capturing fine detail and sharp edges.

Why CT (image domain) instead of CBCT (projection domain)?

This paper works entirely on the final reconstructed CT slices — the kind of 2D cross-section image a radiologist looks at, 512x512 pixels, measured in HU. They’re not touching raw projection data at all.

How they got training data: Similar problem to Paper 1 — they don’t have many real CT scans *with* metal AND a perfect ground-truth mask. So: 1. They took 100 real patients’ CT scans that have *no* metal artifacts (clean scans from radiotherapy planning). 2. They took a separate library of 100 real metal shapes (dental fillings, screws, hip implants, coils, wires) collected from another research group’s dataset. 3. They digitally “pasted” a randomly chosen metal shape into each clean CT image, and simulated the realistic artifacts (streaks, noise) that metal would cause, using a known simulation method. 4. Now they have a (CT image with artificial metal artifact, exact metal mask) pair for training — 11,280 slices for training, 2,820 for testing. 5. They also tested on *real* clinical scans with actual metal implants, a

public dataset called CTPelvic1K, and physical “phantoms” (manufactured test objects with known titanium rods, used to verify accuracy against a known ground truth).

The two “secret sauce” components of DiffSeg:

- **Conditional Dynamic Coding:** At every denoising step, the model combines information from (a) the current, partially-denoised mask and (b) the original CT image, at multiple zoom levels. This helps the model stay “anchored” to where the metal actually is in the real image, instead of drifting off.
- **GFParse (Global Frequency Parser):** A learnable filter that operates in “frequency space” (using something called FFT — Fast Fourier Transform, a standard signal-processing tool that separates an image into smooth, low-frequency parts and sharp, high-frequency parts). This filter removes unwanted high-frequency noise that gets introduced by the fusion step above, resulting in cleaner, less jagged mask edges.

Headline numbers (Slide 7 stat cards):

- **95.45% mean DSC** — DiffSeg’s score on the 2,820 simulated test slices, the highest of all models they compared (including plain U-Net, Attention U-Net, R2U-Net, and DeepLabV3+).
- **97.89% accuracy** — the percentage of all pixels (metal + background) correctly classified. Note this number is naturally high because most pixels in any CT slice are background, not metal — DSC is the more meaningful metric for the metal region itself.
- **84.19% mean DSC for “T3000”** — the simple old-fashioned method: just threshold the image at 3000 HU and call everything above that “metal.” DiffSeg beats this by over 11 percentage points.
- **93.52% mean DSC for plain U-Net** — even a standard U-Net (without DiffSeg’s special tricks) does reasonably well, but DiffSeg is still meaningfully better (~2 points), and the difference was shown to be statistically significant.

Takeaway sentence: “DiffSeg takes the newer ‘diffusion model’ idea — normally used for AI image generation — and applies it to the *segmentation* task, with two extra components that keep it grounded in the real image and clean up noisy edges. It outperformed every traditional deep learning baseline they tested, on simulated data, real clinical scans, and physical phantom test objects.”

Slide 8 — Paper 2 DiffSeg Architecture

Step 1 — Artifact Simulation: As described above — paste a real metal shape into a clean CT slice and simulate the resulting artifact (streaks, extra noise) so the training image looks like a realistic metal-affected scan.

Step 2 — Diffusion Process: The forward/reverse noise process described above. “T=1000” means there are 1000 small noise steps — but during actual use (inference), they use a shortcut method called “DPM-Solver” that gets a similar result in only about 100 steps, making it faster.

Step 3 — Dynamic Conditional Coding: The fusion mechanism that combines the current denoising step’s features with the original CT image’s features, at several different resolutions (“multiple scales”) — like checking your work against the original photo at both a zoomed-out and zoomed-in view.

Step 4 — GParser: The Fourier-based noise filter described above — think of it as a “smart noise-reduction” step specifically for the mask features, applied using frequency-domain math.

Bottom black bar — Ablation results: An “ablation study” means: “what happens if we remove each piece of our method, one at a time, to see how much it actually contributes?” - Remove BOTH special components -> DSC drops to 93.56% (“DiffSeg_1”) - Keep only Dynamic Conditional Coding (remove GParser) -> DSC = 94.35% (+0.79% improvement from adding conditional coding alone) - Keep both (full DiffSeg) -> DSC = 95.45% (+1.1% total improvement)

This proves both add-on components genuinely help — it’s not just marketing, the numbers go up step by step as each piece is added back in.

Slide 9 — Head-to-Head Comparison Table

Walk through each row:

- **Modality/Data:** Paper 1 used CBCT (cone-beam, common in dental/extremity imaging — knees, wrists, ankles, feet). Paper 2 used regular CT (whole-body scanners — head/neck, chest, abdomen) plus physical phantoms.
 - **Model:** Paper 1 = a fairly standard U-Net (CNN = “Convolutional Neural Network,” the basic building block of most image AI). Paper 2 = a newer, more complex diffusion model built on a “ResU-Net” (U-Net with “ResNet”- style blocks, another well-known architecture upgrade).
 - **Training data:** Both papers solve the “no good ground truth” problem the same way — by simulating realistic fake data — but the simulation approaches differ (full physics photon simulation vs. inserting real metal shapes + simulated artifact noise).
 - **Best DSC:** 94.8% vs 95.45% — very close! Both approaches work well.
 - **Best Accuracy/IOU:** Different metrics were emphasized by each paper, but both indicate strong performance.
 - **vs Threshold baseline:** Both papers show deep learning clearly beats the old “just pick a brightness threshold” approach — but interestingly, Paper 2’s gap over its threshold baseline (84.19% -> 95.45%, +11 points) is much bigger than Paper 1’s gap (90.5% -> 94.8%, +4 points). This is partly because **simple thresholding works reasonably well on clean reconstructed CT images** (image domain) but works poorly on raw, noisy projections (projection domain) — so Paper 1’s “easy baseline” (CMAR) was already decent, while Paper 2’s threshold baseline had more room for the AI to improve on.
 - **Key strength:** This is the heart of the comparison — see Slide 10.
 - **Inference cost:** Paper 1’s U-Net produces a result in a single quick pass (milliseconds). Paper 2’s diffusion model needs to run its denoising loop (~3 seconds per batch even with the speed-up). For a hospital wanting fast results on hundreds of images, this matters.
-

Slide 10 — Verdict: Which Domain Works Better?

The honest, nuanced answer: Neither one is a strict “winner” overall — they’re optimized for different situations, which is normal in research.

Why projection domain (Paper 1) is the practical choice for CBCT/MAR pipelines: - If metal sits partly outside the reconstructed area (out-of-FOV), an image-domain method literally cannot see it — there’s nothing there in the 3D image to threshold. But it’s still visible in the raw 2D projections, so a projection-domain model can still find it. - If the patient moved during the scan (motion artifact), the 3D reconstruction gets blurry and unreliable, making image-domain segmentation and the “forward projection” step error-prone. Projection-domain segmentation skips this problem entirely. - It slots directly into existing MAR pipelines without an extra forward-projection conversion step. - Single fast pass through the network — good for real-time or high-throughput use.

Why image domain (DiffSeg, Paper 2) wins on raw segmentation quality: - Highest DSC (95.45%) and accuracy (97.89%) of everything tested in either paper. - The GFParser component specifically targets clean, sharp mask edges — useful when later steps (like NMAR — Normalized MAR) need a tight, accurate mask to preserve as much real bone/tissue detail as possible right next to the metal. - If you have the compute budget for diffusion sampling (a few seconds per image is fine for your use case), it gives you the best possible mask.

One sentence to tie it together: “If your priority is robustness in a real CBCT clinical pipeline — especially with motion or out-of-FOV metal — go projection-domain. If your priority is the sharpest, most accurate mask on already-reconstructed CT images and you can afford slightly slower inference, the diffusion-based image-domain approach (DiffSeg) currently gives the best numbers.”

Slide 11 — My Implementation (Notebook)

This is YOUR work — explain it confidently, it’s simpler than both papers above but uses the exact same core idea (U-Net for binary segmentation).

What the notebook does, step by step: 1. **Loads data:** 500 CT slices (512x512 pixels) and their matching metal masks, stored as raw float32 binary files (a very basic file format — just numbers, no image headers). 2. **Preprocessing:** - Normalizes the image brightness to a 0-1 range (min-max normalization) — this just rescales pixel values so the network trains more smoothly, regardless of the original HU range. - Resizes everything down to 128x128 pixels (much smaller than the 512x512 original) — this makes training much faster on limited hardware, at the cost of losing some fine detail (you note this as a limitation, which mirrors what both papers discuss about preserving detail near metal edges). - Uses *different* resizing methods for the image (smooth/“bilinear”) vs. the mask (nearest-neighbour) — this is important because masks are binary (0 or 1); smooth resizing would create blurry “in-between” values that don’t make sense for a

yes/no mask. 3. **Model — a small U-Net:** - 2 “encoder” steps that shrink the image while increasing feature depth (16 channels -> 32 channels), plus a “bottleneck” (64 channels) at the smallest size. - 2 “decoder” steps that expand back up to full size, each one combining (“concatenating”) with the matching encoder step’s features via skip connections — exactly the U-Net pattern from the original 2015 paper. - Final 1x1 convolution + sigmoid activation turns the last feature map into a single probability-per-pixel output (0 = background, 1 = metal), same idea as both research papers. 4. **Training:** Split data 80% train / 20% test. Used the Adam optimizer (a standard, reliable training algorithm) and binary cross-entropy loss (same loss function as Paper 1) for 15 epochs (= 15 full passes over the training data).

Results to mention: - Training accuracy reached 99.86%, validation accuracy 99.89% by the final epoch, with loss steadily decreasing from ~0.61 to ~0.004 — meaning the model learned successfully and didn’t overfit (training and validation numbers stayed close together). - The sample images show: the original CT slice, the true (ground-truth) metal mask, and the model’s predicted mask — visually, the prediction correctly finds both metal regions, though it’s slightly larger/blurrier than the hand-drawn ground truth.

Important context for accuracy numbers: That ~99.8% accuracy figure sounds amazing, but it’s a bit misleading on its own — in any CT slice, the vast majority of pixels are background (not metal), so a model that just guessed “background everywhere” would already score very high on plain accuracy. This is exactly why both papers (and your “next steps”) emphasize DSC/IOU — metrics that specifically measure how well the *metal* regions were found, not just overall pixel correctness.

Slide 12 — Observations & Next Steps

Observations (left card) — explain each in your own words: - The high accuracy number needs the DSC/IOU caveat explained above. - “Predicted mask is smoother/larger than ground truth” — this is a very common pattern (Paper 2 calls this “over-segmentation”) — the model is erring on the side of marking a bit more area as metal than strictly necessary, similar to what simple threshold methods do in Paper 2’s comparisons. - “128x128 downsampling may blur small/thin metal structures” — both papers specifically avoided shrinking their images near metal regions, because thin metal objects (like wires or screw threads) can disappear or get distorted when an image is shrunk. - The small 2-level U-Net was “enough” because, in this dataset, the metal regions are large, solid, round blobs that are easy to tell apart from the background — a simpler model can handle “easy” shapes; harder, more complex metal shapes (like Paper 2’s spinal screws) would likely need a deeper network.

Next Steps (right card) — frame these as “how I’d extend this toward the rigor of the two papers”: - Compute DSC/IOU/sensitivity/specificity (the same four metrics Paper 2 uses) so results can be directly compared to published numbers. - Compare against a simple HU threshold (2500 or 3000 HU, as in Paper 2) to show how much the deep learning model actually improves over the “naive” baseline. - Add data augmentation (flipping, rotating, elastic deformation) — this is literally the technique from the original U-Net paper (Section 3.1, “Data Augmentation”) and from Paper 1’s training

strategy, and it helps a model generalize from a limited dataset. - Avoid downsampling, or at least test on full-resolution crops, to preserve fine metal boundary detail. - Long-term: try extending this work into the *projection domain* (like Paper 1) using simulated CBCT data — this would be the natural “level up” project, combining everything learned from both papers.

Slide 13 — Summary

This is your closing slide — read through the four bullet points naturally; they’re a condensed recap of everything above: 1. Deep learning clearly beats simple HU-threshold segmentation in both domains (~84% -> ~95% DSC). 2. Projection-domain (Paper 1) is the more practical choice for real CBCT/MAR pipelines, especially for motion and out-of-FOV cases. 3. Image-domain DiffSeg (Paper 2) achieves the best raw segmentation numbers via its diffusion + frequency-filtering approach. 4. Your notebook is a working image-domain U-Net — a solid foundation that, with the next steps above, could be evaluated with the same rigor as Paper 2.

Quick glossary (keep this open during Q&A)

Term	Plain-English meaning
HU (Hounsfield Unit)	A brightness scale for CT images that corresponds to material density (air ~ -1000, water = 0, bone ~500-1500, metal > 3000)
Reconstruction / FDK	The math that turns many 2D X-ray projections into a 3D image
Projection	A single raw 2D X-ray image taken at one angle, before reconstruction
Sinogram	A 2D arrangement of all projection data for one CT slice (used in CT, similar idea to “projections” in CBCT)
Segmentation	Drawing a mask that marks which pixels belong to an object (here: metal)
MAR (Metal Artifact Reduction)	The overall process of fixing images affected by metal
DSC (Dice Score)	Overlap score between predicted and true mask (0-100%, higher = better)
IOU (Intersection over Union)	Another overlap score, slightly stricter than DSC
CNN	Convolutional Neural Network — the general family of AI models used for images
U-Net	A specific CNN shape for segmentation: shrink then

	expand, with skip connections
Diffusion model	An AI model that learns to remove noise step-by-step to generate/refine an output
Monte Carlo simulation	A physics-based simulation that models random events (here: X-ray photons) to generate realistic synthetic data
Ablation study	An experiment where you remove parts of a method one at a time to see how much each part helps
Out-of-FOV (Field of View)	Part of the patient/metal that falls outside the area the scanner reconstructs
Epoch	One full pass of the training data through the model during training
